

Trabajo Final Integrador (TFI)

Materia: Desarrollo de Aplicaciones Web

"Del Código a la Arquitectura: El Criterio del Desarrollador"

1. Introducción y Objetivo de la Materia

El objetivo de este Trabajo Final Integrador es consolidar el perfil del **Desarrollador de Software**, distanciándolo del rol de "programador a secas". No se evaluará únicamente la escritura de código funcional, sino la **capacidad crítica para tomar decisiones tecnológicas**, diseñar arquitecturas escalables, planificar infraestructura en la nube y justificar cada componente elegido bajo criterios de negocio, costos y rendimiento.

La libertad tecnológica es total: pueden elegir el *stack* (Node.js, Go, Python, .NET, Java, etc.) que consideren adecuado, pero **cada elección debe estar estrictamente justificada**.

2. Estructura del Proyecto: Fases de Desarrollo

El trabajo se divide en 5 fases incrementales que reflejan la evolución natural de un producto de software en el mundo real.

Fase 1: Arquitectura Base y Esqueleto (El MVP)

- **Definición de la Temática:** El alumno deberá elegir una problemática de software simple (ej: un sistema de reservas, un e-commerce básico, gestión de biblioteca, etc.) y explicar brevemente el modelo de negocio.
- **Planteo Arquitectónico Inicial:** Proponer una arquitectura tradicional (ej: Monolito tradicional, Capas o MVC) que resuelva el problema de forma óptima para un volumen bajo de usuarios.
- **Justificación:** Responder a: ¿Por qué esta arquitectura es suficiente para arrancar? ¿Cuáles son sus límites?
- **Artefactos Requeridos:**
 - Diagrama de arquitectura inicial.
 - Repositorio Git con la estrategia de ramas definida. Captura de pantalla de los commits principales que muestren **únicamente el esqueleto/boilerplate** configurado (sin lógica de negocio aún).

Fase 2: Evolución Forzada a Microservicios

- **El Cambio de Contexto:** El alumno debe plantear un escenario de negocio (ej: pico masivo de tráfico, necesidad de equipos independientes) que *demande* migrar a microservicios.
- **Paso A: Microservicios Tradicionales:** Diseñar una arquitectura de microservicios básica (comunicación HTTP/REST síncrona, bases de datos compartidas o separadas).
- **Paso B: Microservicios Modernos:** Evolucionar el diseño hacia una arquitectura moderna orientada a eventos (Event-Driven) utilizando Message Brokers (RabbitMQ, Kafka, AWS SQS).
- **Artefactos Requeridos:**

- Diagramas de ambas arquitecturas (Tradicional vs. Moderna), detallando los patrones utilizados (API Gateway, CQRS, Circuit Breaker, Saga, según aplique).
- Captura de los primeros commits del esqueleto de los microservicios interconectados.

Fase 3: Mejora Incremental (Robustez y Escalabilidad)

- **Optimización del Sistema:** Explicar cómo cambiaría la arquitectura de microservicios para soportar alta disponibilidad y tolerancia a fallos a gran escala.
- **Estrategias a incluir:** Implementación de capas de Caché (Redis/Memcached), estrategias de replicación de bases de datos (Read Replicas), balanceadores de carga y políticas de auto-scaling.

Fase 4: Infraestructura, DevOps y Ambientes

Diseñar la infraestructura necesaria para desplegar la arquitectura de la Fase 3 en la Nube (AWS, Azure, GCP o similar).

- **Estructura de Ambientes:** Detallar la separación de los entornos de:
 - **Desarrollo (Desa):** Entorno local y sandbox.
 - **Testing/QA:** Pruebas funcionales.
 - **UAT (User Acceptance Testing):** Pruebas de humo y pre-producción.
 - **Producción (Prod):** Entorno vivo con usuarios reales.
- **Estrategia de Branching:** Explicar el flujo de Git elegido (GitFlow, GitHub Flow, Trunk-Based Development) y cómo se mapea con cada ambiente.
- **Automatización (Pipelines CI/CD):** Diseñar el flujo de integración y despliegue continuo. ¿Qué herramientas se usan? (GitHub Actions, GitLab CI, Jenkins).
- **Contenedores y Orquestación:** Definir el uso de **Docker** para la contenerización y **Kubernetes** para la orquestación (Pods, Deployments, Services).
- **Infraestructura como Código (IaC):** Definir cómo se estructurarían los archivos de **Terraform** para levantar estos servidores de forma automatizada.
- **Estimación de Costos y Servidores:** Presentar una tabla con la cantidad de servidores/nodos necesarios y un costo estimado mensual aproximado del ecosistema en la nube elegida.

Componente	Servicio Nube (Ej: AWS)	Cantidad	Rol en la Arquitectura	Costo Mensual Aprox (USD)
API Gateway	AWS API Gateway	1	Enrutamiento de entrada	\$XX.XX

Microservicios	EKS (Kubernetes)	3 Nodos	Cómputo de la App	\$XX.XX
Base de Datos	RDS PostgreSQL	1 Master / 1 Replica	Persistencia de datos	\$XX.XX

Fase 5: Extensión de Próxima Generación (Integración con IA)

Elevar la arquitectura al siguiente nivel tecnológico. El alumno debe explicar cómo se transformaría su sistema si el negocio exigiera:

- **Opción A: Conexión a APIs de LLM:** Consumo de modelos (OpenAI, Anthropic, etc.) para tareas de clasificación, resúmenes o agentes dentro del sistema, detallando el manejo de asincronismo y timeouts.
- **Opción B: Arquitectura RAG (Retrieval-Augmented Generation):** Integración de una base de datos vectorial (Pinecone, Milvus, Chroma) para alimentar al LLM con datos propios de la aplicación. Mostrar en el diagrama el flujo de *embedding* e ingesta de datos.

3. Entregables Obligatorios

Para aprobar la materia, cada alumno o equipo deberá presentar tres piezas fundamentales:

1. Informe Final (Documento Técnico)

Un documento PDF riguroso que contenga el desarrollo de las 5 fases. Cada decisión tomada debe tener una sección de **"Justificación Técnica y de Negocio"**. Debe incluir los enlaces a los repositorios de código y las capturas de pantalla de los commits del esqueleto solicitado.

2. Presentación Ejecutiva (10 Minutos)

Defensa oral del proyecto simulando una venta técnica a un Director de Tecnología (CTO) o Comité de Arquitectura.

- **Regla estricta:** No se puede mostrar código. La presentación debe enfocarse en los diagramas de arquitectura, la evolución del sistema, la infraestructura y la justificación de costos/decisiones.

3. Artículo Técnico (Divulgación)

Un artículo escrito en formato de blog (estilo Medium, Dev.to o LinkedIn Article) donde se relate la experiencia en primera persona. Debe abordar:

- Los principales desafíos arquitectónicos encontrados.
- Los aprendizajes clave del paso de "programar" a "diseñar".
- En qué escenarios reales de la industria aplicarían la arquitectura final lograda.

4. Criterios de Evaluación

- **Criterio Arquitectónico (40%):** Capacidad de justificar por qué se usa un patrón o una tecnología sobre otra. No sirve el "la uso porque me gusta".
- **Diseño de Infraestructura y DevOps (30%):** Correcta separación de ambientes, entendimiento de la nube, Kubernetes, Terraform y coherencia en la estimación de costos.
- **Evolución e Integración de IA (15%):** Correcto planteo conceptual del flujo de datos con LLMs o arquitecturas RAG.
- **Calidad de los Entregables (15%):** Claridad en el informe, capacidad de síntesis en la presentación de 10 minutos y habilidades de comunicación en el artículo.